Codilar_Testimonial: line-by-line guide

Magento 2 module at `app/code/Codilar/Testimonial`: 19 files, explained line by line.

1. What the module does

Codilar_Testimonial adds a public page at `/testimonial` where visitors can read customer testimonials (newest first, 10 per page, with star ratings) and submit their own using a form. A "Testimonials" link is added to the footer of every page, and three sample testimonials are inserted on install.

2. Files by layer

Layer	Files	Role
Registration	registration.php, etc/module.xml	Tell Magento the module exists and its load order
Routing	etc/frontend/routes.xml	Map `/testimonial/*` URLs to this module
Database	etc/db_schema.xml, etc/db_schema_whitelist.json	Create the `custom_testimonial` table
Seed data	Setup/Patch/Data/AddDefaultTestimonials.php	Insert three sample rows on install
Model	Model/Testimonial.php, Model/ResourceModel/Testimonial.php, .../Testimonial/Collection.php	One row, DB access, many rows
Controllers	Controller/Index/Index.php, Controller/Index/Save.php	Show the page (GET), handle the form (POST)
Blocks	Block/Form.php, Block/TestimonialList.php	Supply data to the templates
Layout	view/frontend/layout/default.xml, testimonial_index_index.xml	Footer link; place blocks on the page
Templates	view/frontend/templates/form.phtml, list.phtml	HTML output
Styles	view/frontend/web/css/source/_module.less	Card grid, star rating, honeypot hiding
Translations	i18n/en_US.csv	Translatable text

3. Request flow

Viewing the page (GET /testimonial)

1. `routes.xml` maps `testimonial` to this module; with no controller or action in the URL, `index/index` is used.
2. `Controller/Index/Index.php` creates a page and sets its title.
3. Magento loads the layout handles `default` (footer link) and `testimonial_index_index` (form and list blocks).
4. `Block/Form` renders `form.phtml`; `Block/TestimonialList` loads the collection, adds a pager and renders `list.phtml`.
5. `_module.less`, compiled into the theme CSS, styles the result.

Submitting a testimonial (POST /testimonial/index/save)

1. `Controller/Index/Save.php` checks the form key (CSRF), then the honeypot (spam).
2. It cleans the input (trim, strip HTML) and validates name, city, rating (1 to 5) and message (20 to 1000 characters).
3. **Invalid:** it shows error messages, stores the input in the session (DataPersistor) and redirects to `/testimonial`, where `Block/Form` refills the fields once.
4. **Valid:** it creates a model with `TestimonialFactory`, saves it through the resource model, shows a thank-you message and redirects to `/testimonial`.

4. Database table `custom_testimonial`

Column	Type	Notes
testimonial_id	INT UNSIGNED, auto increment	Primary key
name	VARCHAR(255) NOT NULL	Required
city	VARCHAR(100) NULL	Optional
rating	SMALLINT UNSIGNED NOT NULL	1 to 5, enforced in PHP
message	TEXT NOT NULL	20 to 1000 characters, enforced in PHP
created_at	TIMESTAMP, default CURRENT_TIMESTAMP	Indexed; used for sorting

5. Classes you will not find in the folder

`TestimonialFactory` and `CollectionFactory` are referenced but have no source file. Magento generates them automatically in `generated/code/Codilar/Testimonial/...` the first time they are needed (or during `bin/magento setup:di:compile`). A factory's only job is `create()`, which returns a new, empty object each time.

6. How to read the rest of this document

Each file below has a short summary followed by a table. The left column gives the line numbers, the middle column shows the exact code from those lines, and the right column explains what they do. Closely related lines are grouped together, and every line of every file is covered.

registration.php

> The entry point of every Magento module. Without it Magento does not know the module exists.

Lines	Code	Explanation
1	`<?php`	Opens a PHP block. The file has no closing `?>`, which is the PHP convention for code-only files (it avoids accidental whitespace output).
2	`use Magento\Framework\Component\ComponentRegistrar;`	Imports `ComponentRegistrar`, the framework class that keeps the list of all modules, themes, languages and libraries.
3		Blank line for readability.
4	`ComponentRegistrar::register(ComponentRegistrar::MODULE, 'Codilar_Testimonial', __DIR__);`	Registers a component of type `MODULE` named `Codilar_Testimonial` whose files live in `__DIR__` (this folder). Magento finds this file through `app/etc/NonComposerComponentRegistration.php`, which globs `app/code/*/*/registration.php` on every request.

`etc/module.xml`

> Declares the module to Magento and its load order. Enabling it with `bin/magento module:enable` writes it to `app/etc/config.php`.

Lines	Code	Explanation
1	`<?xml version="1.0"?>`	Standard XML declaration.

2-3	`<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"`
	
	`xsi:noNamespaceSchemaLocation="urn:magento:framework:Module/etc/module.xsd">`
		Root `<config>` element. `xsi:noNamespaceSchemaLocation` points to the XSD Magento uses to validate this file, so typos are caught at compile time.

4	`    <module name="Codilar_Testimonial">`	Declares the module name. It must match the name used in `registration.php` (`Vendor_Module`). There is no `setup_version` because the module uses declarative schema and data patches instead of old-style install scripts.

5-7	`        <sequence>`
	`            <module name="Magento_Theme"/>`
	`        </sequence>`
		`<sequence>` tells Magento to load `Magento_Theme` before this module. That matters because this module references the `footer_links` block and the `Pager` block, both from Magento_Theme, and its layout must be merged after Theme's.

8-9	`    </module>` `</config>`	Close the `module` and `config` elements.

`etc/frontend/routes.xml`

> Maps a URL prefix to this module's controllers. It sits in `etc/frontend/`, so the route only exists on the storefront, not in the admin.

Lines	Code	Explanation

1-3	`<?xml version="1.0"?>`
	`<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"`
	
	`xsi:noNamespaceSchemaLocation="urn:magento:framework:App/etc/routes.xsd">`
		XML declaration and root element validated against `routes.xsd`.

4	`    <router id="standard">`	`router id="standard"` is the normal storefront router that resolves URLs of the form `/frontName/controller/action`.
5	`        <route id="testimonial" frontName="testimonial">`	Defines a route with internal id `testimonial` and URL prefix (`frontName`) `testimonial`. The route id is also the first part of layout handle names, which is why the page layout file is called `testimonial_index_index.xml`.
6	`            <module name="Codilar_Testimonial"/>`	Says that controllers for this route live in `Codilar_Testimonial`. So `/testimonial/index/save` resolves to `Controller/Index/Save.php`, and plain `/testimonial` defaults to `index/index`, which is `Controller/Index/Index.php`.

7-9	`        </route>`
	`    </router>`
	`</config>`
		Close the elements.

etc/db_schema.xml

> Declarative schema: describes the table this module needs. On `bin/magento setup:upgrade` Magento compares this file with the real database and creates or alters the table to match.

Lines	Code	Explanation

1-3	`<?xml version="1.0"?>`
	`<schema xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"`
	
	`xsi:noNamespaceSchemaLocation="urn:magento:framework:Setup/Declaration/Schema/etc/schema.xsd">`
		XML declaration and root `<schema>` element validated against Magento's declarative schema XSD.

4	`    <table name="custom_testimonial" resource="default" engine="innodb" comment="Customer Testimonials">`	Declares table `custom_testimonial` on the `default` DB connection, using the InnoDB engine (transactions, row locking), with a table comment.
5-6	`        <column xsi:type="int" name="testimonial_id" unsigned="true" nullable="false" identity="true"` `                comment="Testimonial ID"/>`	`testimonial_id`: an unsigned `INT`, NOT NULL, with `identity="true"`, which means AUTO_INCREMENT. It is the row's unique id.
7	`        <column xsi:type="varchar" name="name" length="255" nullable="false" comment="Customer Name"/>`	`name`: `VARCHAR(255)`, required. The customer's name.
8	`        <column xsi:type="varchar" name="city" length="100" nullable="true" comment="City"/>`	`city`: `VARCHAR(100)`, nullable, so the city is optional.
9	`        <column xsi:type="smallint" name="rating" unsigned="true" nullable="false" comment="Rating (1-5)"/>`	`rating`: unsigned `SMALLINT`, required. The 1 to 5 range is enforced in PHP (the Save controller), not by the database.
10	`        <column xsi:type="text" name="message" nullable="false" comment="Message"/>`	`message`: `TEXT`, required. The testimonial body.
11-12	`        <column xsi:type="timestamp" name="created_at" on_update="false" nullable="false"` `                default="CURRENT_TIMESTAMP" comment="Created At"/>`	`created_at`: a `TIMESTAMP` that defaults to `CURRENT_TIMESTAMP`, so MySQL fills it in on insert. `on_update="false"` means it is *not* changed when the row is later updated.

13-15	`        <constraint xsi:type="primary" referenceId="PRIMARY">`
	`            <column name="testimonial_id"/>`
	`        </constraint>`
		Primary key constraint on `testimonial_id`. `referenceId` is the name Magento uses to track this constraint.

16-18	`        <index referenceId="CUSTOM_TESTIMONIAL_CREATED_AT" indexType="btree">`
	`            <column name="created_at"/>`
	`        </index>`
		B-tree index named `CUSTOM_TESTIMONIAL_CREATED_AT` on `created_at`. The list page sorts by this column, so the index makes that sort fast.

19-20	`    </table>` `</schema>`	Close the table and schema elements.

etc/db_schema_whitelist.json

> A safety list generated by `bin/magento setup:db-declaration:generate-whitelist --module-name=Codilar_Testimonial`. Magento only drops a column, index or constraint if it appears here, so it never deletes database objects this module did not create.

Lines	Code	Explanation
1-2	`{` `  "custom_testimonial": {`	Opens the JSON object and the entry for table `custom_testimonial`.

3-10	`  "column": {`
	`  "testimonial_id": true,`
	`  "name": true,`
	`  "city": true,`
	`  "rating": true,`
	`  "message": true,`
	`  "created_at": true`
	`  },`
		Every column this module has declared. If one is later removed from `db_schema.xml`, Magento knows it is allowed to drop it.

11-13	`  "index": {`
	`  "CUSTOM_TESTIMONIAL_CREATED_AT": true`
	`  },`
		The index this module owns.

14-16	`  "constraint": {`
	`  "PRIMARY": true`
	`  }`
		The primary key constraint this module owns.

17-18	`  }` `}`	Close the objects. Keep this file in sync by regenerating it whenever `db_schema.xml` changes.

Model/Testimonial.php

> The **model**: one object = one testimonial row. It holds data and delegates loading and saving to the resource model.

Lines	Code	Explanation
1-2	`<?php` `declare(strict_types=1);`	Opens PHP and enables strict typing: PHP throws a `TypeError` instead of silently converting a wrongly typed argument.
3-4	`namespace Codilar\Testimonial\Model;`	Namespace, which maps to the folder `Codilar/Testimonial/Model` (PSR-4 autoloading).
5-7	`use Codilar\Testimonial\Model\ResourceModel\Testimonial as TestimonialResource;` `use Magento\Framework\Model\AbstractModel;`	Imports the resource model (aliased `TestimonialResource` to avoid a clash with this class's own name) and Magento's `AbstractModel`.

8-11	`/**`
	` * Customer testimonial shown on the /testimonial storefront page.`
	` */`
		Class docblock describing what the class is.

12-13	`class Testimonial extends AbstractModel` `{`	The class extends `AbstractModel`, which provides `getData()`, `setData()`, magic getters and setters, and save/load events.

14-19	`    public const TESTIMONIAL_ID = 'testimonial_id';`
	`    public const NAME = 'name';`
	`    public const CITY = 'city';`
	`    public const RATING = 'rating';`
	`    public const MESSAGE = 'message';`
	`    public const CREATED_AT = 'created_at';`
		Constants for every column name. Other classes use `Testimonial::NAME` rather than the string `'name'`, so a typo becomes a PHP error instead of a silent bug, and renaming a column means changing one place.

20-24	`    /**`
	`     * @var string`
	`     */`
	`    protected $_eventPrefix = 'codilar_testimonial';`
		`$_eventPrefix` sets the prefix of events this model dispatches, for example `codilar_testimonial_save_before`, `codilar_testimonial_save_after` and `codilar_testimonial_load_after`. Other modules can observe these events.

25-28	`    /**`
	`     * @inheritdoc`
	`     */`
		Docblock: `@inheritdoc` means "same documentation as the parent method".

29-32	`    protected function _construct(): void`
	`    {`
	`        $this->_init(TestimonialResource::class);`
	`    }`
		`_construct()` (one underscore) is Magento's pseudo-constructor, called at the end of the real constructor. `_init()` links this model to its resource model class, which is what knows the table name.

33	`}`	End of class.

Model/ResourceModel/Testimonial.php

> The **resource model**: the only class that talks to the database table for a single record (insert, update, delete, load by id).

Lines	Code	Explanation

1-4	`<?php`
	`declare(strict_types=1);`
	
	`namespace Codilar\Testimonial\Model\ResourceModel;`
		Strict types and the namespace `...\Model\ResourceModel`.

5-7	`use Magento\Framework\Model\ResourceModel\Db\AbstractDb;`	Imports `AbstractDb`, the base class that implements `save()`, `load()` and `delete()` with SQL.
8-9	`class Testimonial extends AbstractDb` `{`	The class has the same short name as the model but lives in a different namespace. This is the normal Magento convention.
10	`    public const TABLE_NAME = 'custom_testimonial';`	Public constant with the table name. The data patch reuses it so the name is written down only once in PHP.

11-14	`    /**`
	`     * @inheritdoc`
	`     */`
		Docblock.

15-18	`    protected function _construct(): void`
	`    {`
	`        $this->_init(self::TABLE_NAME, 'testimonial_id');`
	`    }`
		Pseudo-constructor. `_init(table, idField)` sets the main table and its primary key. `save()` inserts when the id is empty and updates when it is set.

19	`}`	End of class.

Model/ResourceModel/Testimonial/Collection.php

> The **collection**: loads many rows at once and gives back a list of `Testimonial` models. Supports filters, sorting and paging.

Lines	Code	Explanation

1-4	`<?php`
	`declare(strict_types=1);`
	
	`namespace`
	`Codilar\Testimonial\Model\ResourceModel\Testimonial;`
		Strict types. By convention the namespace is the resource model's class name plus `\Testimonial`.

5-9	`use`
	`Codilar\Testimonial\Model\ResourceModel\Testimonial`
	`as TestimonialResource;`
	`use Codilar\Testimonial\Model\Testimonial;`
	`use`
	`Magento\Framework\Model\ResourceModel\Db\Collection\`
	`AbstractCollection;`
		Imports the resource model, the model and Magento's `AbstractCollection`.

10-11	`class Collection extends AbstractCollection` `{`	Class declaration.

12-15	`    /**`
	`     * @var string`
	`     */`
	`    protected $_idFieldName = 'testimonial_id';`
		`$_idFieldName` tells the collection which column is the unique id. Items are keyed by it, and methods like `getAllIds()` use it.

16-19	`    /**`
	`     * @inheritdoc`
	`     */`
		Docblock.

20-23	`    protected function _construct(): void`
	`    {`
	`        $this->_init(Testimonial::class,`
	`TestimonialResource::class);`
	`    }`
		`_init(modelClass, resourceModelClass)` tells the collection which model to create for each row and which resource model (and so which table) to read from.

24	`}`	End of class.

`Controller/Index/Index.php`

> Handles **GET /testimonial** (which is `/testimonial/index/index`). It only builds the page; the layout XML decides what is on it.

Lines	Code	Explanation

1-4	`<?php`
	`declare(strict_types=1);`
	
	`namespace Codilar\Testimonial\Controller\Index;`
		Strict types and namespace. The folder `Controller/Index` is the controller part of the URL, and the class name `Index` is the action part.

5-9	`use Magento\Framework\App\Action\HttpGetActionInterface;`
	`use Magento\Framework\View\Result\Page;`
	`use Magento\Framework\View\Result\PageFactory;`
		Imports. `HttpGetActionInterface` marks the action as reachable only by GET (and HEAD). `PageFactory` creates a full HTML page result.

10-13	`/**`
	` * Testimonials page: submission form plus the list of testimonials.`
	` */`
	`class Index implements HttpGetActionInterface`
		Docblock and class. Implementing the interface directly, instead of extending the deprecated `Action` class, is the modern Magento 2.4 style.

14-21	`{`
	`    /**`
	`     * @param PageFactory $pageFactory`
	`     */`
	`    public function __construct(`
	`        private readonly PageFactory $pageFactory`
	`    ) {`
	`    }`
		Constructor property promotion (PHP 8): `private readonly PageFactory $pageFactory` declares and assigns the property in one step. Magento's object manager injects the dependency automatically.

22-26	`    /**`
	`     * @inheritdoc`
	`     */`
	`    public function execute(): Page`
		`execute()` is the method Magento calls when the URL matches. It returns a `Page`.

27-28	`    {` `        $page = $this->pageFactory->create();`	Creates the page result. When rendered, it loads the layout handles `default` and `testimonial_index_index`.
29	`        $page->getConfig()->getTitle()->set(__('Testimonials'));`	Sets the browser tab title (`<title>`) and the on-page heading to "Testimonials". `__()` makes the text translatable.

30-33	`        return $page;`
	`    }`
	`}`
		Returns the page; Magento renders it to HTML.

Controller/Index/Save.php

> Handles **POST /testimonial/index/save**: checks the form key, catches bots, cleans and validates input, saves the testimonial, then redirects back to the list with a message. This is the post/redirect/get pattern, which stops a browser refresh from re-submitting the form.

Lines	Code	Explanation

1-4	`<?php`
	`declare(strict_types=1);`
	
	`namespace Codilar\Testimonial\Controller\Index;`
		Strict types and namespace.

5-16	`use Codilar\Testimonial\Model\ResourceModel\Testimonial as TestimonialResource;`
	`use Codilar\Testimonial\Model\Testimonial;`
	`use Codilar\Testimonial\Model\TestimonialFactory;`
	`use Magento\Framework\App\Action\HttpPostActionInterface;`
	`use Magento\Framework\App\Request\DataPersistorInterface;`
	`use Magento\Framework\App\RequestInterface;`
	`use Magento\Framework\Controller\Result\Redirect;`
	`use Magento\Framework\Controller\Result\RedirectFactory;`
	`use Magento\Framework\Data\Form\FormKey\Validator as FormKeyValidator;`
	`use Magento\Framework\Message\ManagerInterface;`
	`use Psr\Log\LoggerInterface;`
		Imports: the resource model, model and model factory; `HttpPostActionInterface` (POST only); the request; redirect result and factory; form key validator; message manager (the flash messages at the top of the page); data persistor (session storage for form values); and a PSR-3 logger.
		Note: `TestimonialFactory` has no source file in the module. Magento generates it in `generated/code/` the first time it is needed. A factory creates a fresh object every time, unlike normal injection, which shares one instance.

17-22	`/**`
	` * Validates and stores a testimonial submitted from the storefront form.`
	` */`
	`class Save implements HttpPostActionInterface`
	`{`
		Docblock and class. Because it implements `HttpPostActionInterface`, a GET request to this URL returns a 404.

23	`    public const DATA_PERSISTOR_KEY = 'codilar_testimonial_form';`	Session key used to remember what the visitor typed so the form can be refilled after an error. `Block/Form.php` reads the same constant.

24-28	`    /**`
	`     * Hidden field that real visitors never fill in.`
	`     */`
	`    public const HONEYPOT_FIELD = 'website';`
		Name of the honeypot field: a hidden input called `website`. People cannot see it, but simple spam bots fill in every field.

29-33	`    private const MESSAGE_MIN_LENGTH = 20;`
	`    private const MESSAGE_MAX_LENGTH = 1000;`
	`    private const NAME_MAX_LENGTH = 255;`
	`    private const CITY_MAX_LENGTH = 100;`
		Validation limits. They match the database column sizes (255 and 100) and the message rules shown on the form (20 to 1000 characters).

34-55	`    /**`
	`     * @param RequestInterface $request`
	`     * @param RedirectFactory $redirectFactory`
	`     * @param FormKeyValidator $formKeyValidator`
	`     * @param ManagerInterface $messageManager`
	`     * @param DataPersistorInterface $dataPersistor`
	`     * @param TestimonialFactory $testimonialFactory`
	`     * @param TestimonialResource $testimonialResource`
	`     * @param LoggerInterface $logger`
	`     */`
	`    public function __construct(`
	`        private readonly RequestInterface $request,`
	`        private readonly RedirectFactory $redirectFactory,`
	`        private readonly FormKeyValidator $formKeyValidator,`
	`        private readonly ManagerInterface $messageManager,`
	`        private readonly DataPersistorInterface $dataPersistor,`
	`        private readonly TestimonialFactory $testimonialFactory,`
	`        private readonly TestimonialResource $testimonialResource,`
	`        private readonly LoggerInterface $logger`
	`    ) {`
	`    }`
		Constructor with eight dependencies, injected by Magento's object manager. All are `private readonly`, so they cannot be reassigned.

56-61	`    /**`
	`     * @inheritdoc`
	`     */`
	`    public function execute(): Redirect`
	`    {`
		`execute()` always returns a redirect.

62	`        $redirect = $this->redirectFactory->create()->setPath('testimonial');`	Builds a redirect to `/testimonial`. Every path below returns it, so the visitor always ends up on the testimonials page.

63-67	`        if (!$this->formKeyValidator->validate($this->request)) {`
	`            $this->messageManager->addErrorMessage(__('Invalid form key. Please refresh the page and try again.'));`
	`            return $redirect;`
	`        }`
		CSRF protection: checks that the hidden `form_key` sent with the form matches the visitor's session. If not, it shows an error and stops. (Magento's own request validator already checks form keys on frontend POST actions, so this is an extra layer that also gives a friendlier message.)

68-73	`        // Bots fill every field; pretend success so they get no signal.`
	`        if (trim((string) $this->request->getParam(self::HONEYPOT_FIELD)) !== '') {`
	`            $this->messageManager->addSuccessMessage(__('Thank you for your testimonial!'));`
	`            return $redirect;`
	`        }`
		Honeypot check. If the hidden field has any value, the sender is almost certainly a bot. It shows the normal success message without saving anything, so the bot gets no hint that it was blocked.

74-80	`        $data = [`
	`            Testimonial::NAME => $this->clean($this->request->getParam(Testimonial::NAME)),`
	`            Testimonial::CITY => $this->clean($this->request->getParam(Testimonial::CITY)),`
	`            Testimonial::RATING => $this->clean($this->request->getParam(Testimonial::RATING)),`
	`            Testimonial::MESSAGE => $this->clean($this->request->getParam(Testimonial::MESSAGE)),`
	`        ];`
		Reads the four fields from the request and passes each through `clean()` (trim and strip HTML). Uses the model constants as both the request parameter names and the array keys.

81-89	`        $errors = $this->validate($data);`
	`        if ($errors) {`
	`            foreach ($errors as $error) {`
	`                $this->messageManager->addErrorMessage($error);`
	`            }`
	`            $this->dataPersistor->set(self::DATA_PERSISTOR_KEY, $data);`
	`            return $redirect;`
	`        }`
		Runs `validate()`. If there are errors, it shows each one as a flash message, saves the cleaned input in the session through the data persistor (so the form can be refilled) and redirects.

90-99	`        try {`
	`            $testimonial = $this->testimonialFactory->create();`
	`            $testimonial->setData([`
	`                Testimonial::NAME => $data[Testimonial::NAME],`
	`                Testimonial::CITY => $data[Testimonial::CITY] !== '' ? $data[Testimonial::CITY] : null,`
	`                Testimonial::RATING => (int) $data[Testimonial::RATING],`
	`                Testimonial::MESSAGE => $data[Testimonial::MESSAGE],`
	`            ]);`
	`            $this->testimonialResource->save($testimonial);`
		Happy path: creates a new, empty model with the factory and fills it. An empty city is stored as `NULL` rather than `''`, and the rating is cast to an integer. `testimonialResource->save()` runs the SQL INSERT. `created_at` is left out, so the database default fills it in.

100-107	`        } catch (\Exception $e) {`
	`            $this->logger->error($e->getMessage(), ['exception' => $e]);`
	`            $this->messageManager->addErrorMessage(`
	`                __('We could not save your testimonial right now. Please try again later.')`
	`            );`
	`            $this->dataPersistor->set(self::DATA_PERSISTOR_KEY, $data);`
	`            return $redirect;`
	`        }`
		If saving throws an exception (for example, the database is down), it logs the full exception to `var/log/system.log`, shows a generic message (never the technical error), keeps the typed data and redirects.

108-113	`        $this->dataPersistor->clear(self::DATA_PERSISTOR_KEY);`
	`        $this->messageManager->addSuccessMessage(__('Thank you for your testimonial!'));`
	
	`        return $redirect;`
	`    }`
		Success: clears any stored form data, shows "Thank you for your testimonial!" and redirects.

114-124	`    /**`
	`     * Trim and strip HTML tags from a request value.`
	`     *`
	`     * @param mixed $value`
	`     * @return string`
	`     */`
	`    private function clean(mixed $value): string`
	`    {`
	`        return is_scalar($value) ? trim(strip_tags((string) $value)) : '';`
	`    }`
		`clean()`: `is_scalar()` rejects arrays (for example a crafted request with `name[]=x`), which would otherwise break string functions. Otherwise it casts to string, removes HTML tags with `strip_tags()` and trims whitespace.

125-135	`    /**`
	`     * Return a list of validation error messages (empty when the data is valid).`
	`     *`
	`     * @param array $data`
	`     * @return \Magento\Framework\Phrase[]`
	`     */`
	`    private function validate(array $data): array`
	`    {`
	`        $errors = [];`
		`validate()` collects all problems in an array instead of stopping at the first, so the visitor sees every error at once.

136-140	`        if ($data[Testimonial::NAME] === '') {`
	`            $errors[] = __('Please enter your name.');`
	`        } elseif (mb_strlen($data[Testimonial::NAME]) > self::NAME_MAX_LENGTH) {`
	`            $errors[] = __('Name must be %1 characters or fewer.', self::NAME_MAX_LENGTH);`
	`        }`
		Name is required and at most 255 characters. `mb_strlen` counts characters, not bytes, so names in Arabic, Hindi and other scripts are measured correctly.

141-144	`        if (mb_strlen($data[Testimonial::CITY]) > self::CITY_MAX_LENGTH) {`
	`            $errors[] = __('City must be %1 characters or fewer.', self::CITY_MAX_LENGTH);`
	`        }`
		City is optional but at most 100 characters.

145-153	`        $rating = filter_var(`
	`            $data[Testimonial::RATING],`
	`            FILTER_VALIDATE_INT,`
	`            ['options' => ['min_range' => 1, 'max_range' => 5]]`
	`        );`
	`        if ($rating === false) {`
	`            $errors[] = __('Please choose a rating between 1 and 5 stars.');`
	`        }`
		Rating must be a whole number from 1 to 5. `filter_var` with `FILTER_VALIDATE_INT` and a range returns `false` for an empty value, decimals, text or out-of-range numbers.

154-164	`        $messageLength = mb_strlen($data[Testimonial::MESSAGE]);`
	`        if ($messageLength === 0) {`
	`            $errors[] = __('Please enter your message.');`
	`        } elseif ($messageLength < self::MESSAGE_MIN_LENGTH || $messageLength > self::MESSAGE_MAX_LENGTH) {`
	`            $errors[] = __(`
	`                'Your message must be between %1 and %2 characters.',`
	`                self::MESSAGE_MIN_LENGTH,`
	`                self::MESSAGE_MAX_LENGTH`
	`            );`
	`        }`
		Message is required and must be 20 to 1000 characters. Two separate messages make it clear whether the field was empty or just the wrong length.

165-168	`        return $errors;`
	`    }`
	`}`
		Returns the error list (empty means valid) and closes the class.

Block/Form.php

> The **block** (view model) behind `form.phtml`. It gives the template the form's action URL, the honeypot field name and any values to refill after a failed submit.

Lines	Code	Explanation

1-10	`<?php`
	`declare(strict_types=1);`
	
	`namespace Codilar\Testimonial\Block;`
	
	`use Codilar\Testimonial\Controller\Index\Save;`
	`use`
	`Magento\Framework\App\Request\DataPersistorInterface`
	`;`
	`use Magento\Framework\View\Element\Template;`
	`use Magento\Framework\View\Element\Template\Context;`
		Strict types, namespace and imports. It imports the `Save` controller only to reuse its constants, so the session key and honeypot name are defined in one place.

11-15	`/**`
	` * "Write a Testimonial" form; refills the fields`
	`after a failed submission.`
	` */`
	`class Form extends Template`
	`{`
		Extends `Template`, the standard block type that renders a `.phtml` file.

16-19	`    /**`
	`     * @var array|null`
	`     */`
	`    private ?array $formData = null;`
		`$formData` caches the stored form values. `null` means "not read yet".

20-32	`    /**`
	`     * @param Context $context`
	`     * @param DataPersistorInterface $dataPersistor`
	`     * @param array $data`
	`     */`
	`    public function __construct(`
	`        Context $context,`
	`        private readonly DataPersistorInterface`
	`$dataPersistor,`
	`        array $data = []`
	`    ) {`
	`        parent::__construct($context, $data);`
	`    }`
		Constructor. `Context` and `$data` are required by the parent `Template` class and passed up with `parent::__construct()`. The data persistor is promoted to a private readonly property.

33-42	`    /**`
	`     * URL the form posts to.`
	`     *`
	`     * @return string`
	`     */`
	`    public function getFormAction(): string`
	`    {`
	`        return $this-`
	`>getUrl('testimonial/index/save');`
	`    }`
		`getFormAction()` builds the absolute URL `.../testimonial/index/save` that the form posts to.

43-52	`    /**`
	`     * Name of the hidden anti-spam field.`
	`     *`
	`     * @return string`
	`     */`
	`    public function getHoneypotFieldName(): string`
	`    {`
	`        return Save::HONEYPOT_FIELD;`
	`    }`
		`getHoneypotFieldName()` returns `'website'` from the controller constant, so the template's hidden field always matches what the controller checks.

53-61	`    /**`
	`     * Value the visitor previously typed into a field, or an empty string.`
	`     *`
	`     * @param string $field`
	`     * @return string`
	`     */`
	`    public function getFieldValue(string $field): string`
	`    {`
		`getFieldValue($field)`: returns what the visitor typed into a field before a failed submit.

62-65	`        if ($this->formData === null) {`
	`            $this->formData = (array) $this->dataPersistor->get(Save::DATA_PERSISTOR_KEY);`
	`            $this->dataPersistor->clear(Save::DATA_PERSISTOR_KEY);`
	`        }`
		On the first call only: reads the saved array from the session (the `(array)` cast turns `null` into `[]`), then clears it right away. That way the values refill the form once and do not reappear on the next visit.

66-69	`        return (string) ($this->formData[$field] ?? '');`
	`    }`
	`}`
		Returns the field's value as a string, or `''` if it is missing (`??` is the null-coalescing operator). End of class.

Block/TestimonialList.php

> The block behind `list.phtml`. It loads testimonials newest-first, 10 per page, and adds a pager.

Lines	Code	Explanation

1-12	`<?php`
	`declare(strict_types=1);`
	
	`namespace Codilar\Testimonial\Block;`
	
	`use Codilar\Testimonial\Model\ResourceModel\Testimonial\Collection;`
	`use Codilar\Testimonial\Model\ResourceModel\Testimonial\CollectionFactory;`
	`use Codilar\Testimonial\Model\Testimonial;`
	`use Magento\Framework\View\Element\Template;`
	`use Magento\Framework\View\Element\Template\Context;`
	`use Magento\Theme\Block\Html\Pager;`
		Strict types, namespace and imports: the collection and its factory (auto-generated like the model factory), the model (for column constants), `Template`, and Magento's `Pager` block.

13-17	`/**`
	` * Paginated list of testimonials, newest first.`
	` */`
	`class TestimonialList extends Template`
	`{`
		Docblock and class declaration.

18	`    public const PAGE_SIZE = 10;`	`PAGE_SIZE = 10`: testimonials per page.

19-23	`    /**`
	`     * @var Collection|null`
	`     */`
	`    private ?Collection $collection = null;`
		Caches the collection so every caller (template and pager) uses the same object and the query runs only once.

24-36	`    /**`
	`     * @param Context $context`
	`     * @param CollectionFactory $collectionFactory`
	`     * @param array $data`
	`     */`
	`    public function __construct(`
	`        Context $context,`
	`        private readonly CollectionFactory $collectionFactory,`
	`        array $data = []`
	`    ) {`
	`        parent::__construct($context, $data);`
	`    }`
		Constructor: injects the `CollectionFactory` and passes context and data to the parent.

37-44	`    /**`
	`     * Testimonials for the current page.`
	`     *`
	`     * @return Collection`
	`     */`
	`    public function getTestimonials(): Collection`
	`    {`
		`getTestimonials()`: returns the collection for the current page.

45-49	`        if ($this->collection === null) {`
	`            $this->collection = $this->collectionFactory->create()`
	`                ->addOrder(Testimonial::CREATED_AT, Collection::SORT_ORDER_DESC)`
	`                ->addOrder(Testimonial::TESTIMONIAL_ID, Collection::SORT_ORDER_DESC);`
	`        }`
		Created once: newest `created_at` first, with `testimonial_id` descending as a tie-breaker. Seeded rows inserted in the same second share a timestamp, and the tie-breaker keeps their order stable between pages. Nothing is loaded yet; collections are lazy.

50-53	`        return $this->collection;` `    }`	Returns the cached collection.

54-60	`    /**`
	`     * @inheritdoc`
	`     */`
	`    protected function _prepareLayout()`
	`    {`
	`        parent::_prepareLayout();`
		`_prepareLayout()` is a hook Magento calls once the block is added to the layout, before rendering. Calls the parent first.

61-62	`        /** @var Pager $pager */` `        $pager = $this->getLayout()->createBlock(Pager::class, 'codilar.testimonial.pager');`	Creates a `Pager` block named `codilar.testimonial.pager`.

63-67	`        $pager->setLimit(self::PAGE_SIZE)`
	`            ->setAvailableLimit([self::PAGE_SIZE => self::PAGE_SIZE])`
	`            ->setShowPerPage(false)`
	`            ->setShowAmounts(false)`
	`            ->setCollection($this->getTestimonials());`
		Sets the pager up: 10 per page, only one page-size option, no "Show X per page" dropdown, no "Items 1-10 of N" text. `setCollection()` is the important call: the pager reads the `?p=` page number from the URL and applies `LIMIT` / `OFFSET` to the collection.

68-71	`        $this->setChild('pager', $pager);`
	
	`        return $this;`
	`    }`
		Adds the pager as a child block under the alias `pager` and returns `$this`, as the hook requires.

72-81	`    /**`
	`     * Pager HTML (empty when everything fits on one page).`
	`     *`
	`     * @return string`
	`     */`
	`    public function getPagerHtml(): string`
	`    {`
	`        return $this->getChildHtml('pager');`
	`    }`
		`getPagerHtml()` renders the child pager. The page links only show when there is more than one page.

82-92	`    /**`
	`     * Created date formatted for the current locale.`
	`     *`
	`     * @param Testimonial $testimonial`
	`     * @return string`
	`     */`
	`    public function getFormattedDate(Testimonial $testimonial): string`
	`    {`
	`        return $this->formatDate($testimonial->getData(Testimonial::CREATED_AT), \IntlDateFormatter::MEDIUM);`
	`    }`
		`getFormattedDate()` turns the raw `created_at` value into a readable date in the store's locale and timezone, using the `MEDIUM` style (for example "Oct 30, 2026").

93	`}`	End of class.

Setup/Patch/Data/AddDefaultTestimonials.php

> A **data patch**: inserts three sample testimonials the first time `bin/magento setup:upgrade` runs after the module is installed. Magento records applied patches in the `patch_list` table, so it never runs twice.

Lines	Code	Explanation

1-11	`<?php`
	`declare(strict_types=1);`
	
	`namespace Codilar\Testimonial\Setup\Patch\Data;`
	
	`use Codilar\Testimonial\Model\ResourceModel\Testimonial as TestimonialResource;`
	`use Codilar\Testimonial\Model\Testimonial;`
	`use Codilar\Testimonial\Model\TestimonialFactory;`
	`use Magento\Framework\Setup\ModuleDataSetupInterface;`
	`use Magento\Framework\Setup\Patch\DataPatchInterface;`
	`use Magento\Framework\Setup\Patch\PatchRevertableInterface;`
		Strict types, namespace and imports: resource model, model and factory, `ModuleDataSetupInterface` (database access during setup), `DataPatchInterface` and `PatchRevertableInterface`.

12-17	`/**`
	` * Seeds the custom_testimonial table with default testimonials on setup:upgrade.`
	` */`
	`class AddDefaultTestimonials implements DataPatchInterface, PatchRevertableInterface`
	`{`
		Docblock and class. Implementing `PatchRevertableInterface` means the patch can be undone with `bin/magento module:uninstall`.

18-21	`    /**`
	`     * Testimonials to insert. Add or edit rows here.`
	`     */`
	`    private const TESTIMONIALS = [`
		Constant holding the seed rows, with a comment saying where to add or change them. Note that editing it does not re-run an already applied patch; a new patch class is needed for that.

22-27	`        [`
	`            Testimonial::NAME => 'Priya Nair',`
	`            Testimonial::CITY => 'Bengaluru',`
	`            Testimonial::RATING => 5,`
	`            Testimonial::MESSAGE => 'Fast delivery and the product quality exceeded my expectations. Highly recommended!',`
	`        ],`
		Row 1: Priya Nair, Bengaluru, 5 stars.

28-33	`        [`
	`            Testimonial::NAME => 'Ahmed Khan',`
	`            Testimonial::CITY => 'Dubai',`
	`            Testimonial::RATING => 4,`
	`            Testimonial::MESSAGE => 'Great customer support team. They resolved my query within a few hours.',`
	`        ],`
		Row 2: Ahmed Khan, Dubai, 4 stars.

34-40	`        [`
	`            Testimonial::NAME => 'Rahul Menon',`
	`            Testimonial::CITY => 'Kochi',`
	`            Testimonial::RATING => 5,`
	`            Testimonial::MESSAGE => 'Smooth checkout experience and easy returns. I will definitely shop here again.',`
	`        ],`
	`    ];`
		Row 3: Rahul Menon, Kochi, 5 stars. End of the array.

41-52	`    /**`
	`     * @param ModuleDataSetupInterface $moduleDataSetup`
	`     * @param TestimonialFactory $testimonialFactory`
	`     * @param TestimonialResource $testimonialResource`
	`     */`
	`    public function __construct(`
	`        private readonly ModuleDataSetupInterface $moduleDataSetup,`
	`        private readonly TestimonialFactory $testimonialFactory,`
	`        private readonly TestimonialResource $testimonialResource`
	`    ) {`
	`    }`
		Constructor: injects the setup connection, model factory and resource model.

53-58	`    /**`
	`     * @inheritdoc`
	`     */`
	`    public function apply(): self`
	`    {`
		`apply()`: Magento calls this to run the patch.

59-60	`        $connection = $this->moduleDataSetup->getConnection();` `        $connection->startSetup();`	Gets the DB connection and calls `startSetup()`, which turns off foreign key checks and sets the SQL mode for setup work.

61-66	`        foreach (self::TESTIMONIALS as $data) {`
	`            $testimonial = $this->testimonialFactory->create();`
	`            $testimonial->setData($data);`
	`            $this->testimonialResource->save($testimonial);`
	`        }`
		For each seed row: creates a model, fills it and saves it through the resource model, the same way the Save controller does. Using the model means the model's save events fire too.

67-71	`        $connection->endSetup();`
	
	`        return $this;`
	`    }`
		`endSetup()` restores the connection settings; returns `$this`.

72-79	`    /**`
	`     * Remove the testimonials this patch inserted (used by module:uninstall).`
	`     *`
	`     * @return void`
	`     */`
	`    public function revert(): void`
	`    {`
		`revert()`: called on module uninstall to remove the seeded rows. `getTable()` adds any table prefix configured for the installation.

80-89	`        $connection = $this->moduleDataSetup->getConnection();`
	`        $table = $this->moduleDataSetup->getTable(TestimonialResource::TABLE_NAME);`
	
	`        foreach (self::TESTIMONIALS as $data) {`
	`            $connection->delete($table, [`
	`                Testimonial::NAME . ' = ?' => $data[Testimonial::NAME],`
	`                Testimonial::MESSAGE . ' = ?' => $data[Testimonial::MESSAGE],`
	`            ]);`
	`        }`
	`    }`
		For each seed row, deletes rows matching both its name and its message. The `' = ?'` placeholders are bound safely by the database adapter, so this is not open to SQL injection. Testimonials submitted by real customers are left alone.

90-97	`    /**`
	`     * @inheritdoc`
	`     */`
	`    public static function getDependencies(): array`
	`    {`
	`        return [];`
	`    }`
		`getDependencies()`: patches that must run first. None here, because the table comes from `db_schema.xml`, which Magento always applies before data patches.

98-106	`    /**`
	`     * @inheritdoc`
	`     */`
	`    public function getAliases(): array`
	`    {`
	`        return [];`
	`    }`
	`}`
		`getAliases()`: older names this patch was known by. None. End of class.

`view/frontend/layout/default.xml`

> The `default` handle is loaded on **every** storefront page. This file adds a "Testimonials" link to the footer.

Lines	Code	Explanation

1-3	`<?xml version="1.0"?>`
	`<page xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"`
	
	`xsi:noNamespaceSchemaLocation="urn:magento:framework:View/Layout/etc/page_configuration.xsd">`
		XML declaration and the `<page>` root, validated against the page configuration XSD.

4-5	`    <body>` `        <referenceBlock name="footer_links">`	Inside `<body>`, `referenceBlock name="footer_links"` targets the existing footer links block from Magento_Theme. This is why `module.xml` loads after Magento_Theme.
6	`            <block class="Magento\Framework\View\Element\Html\Link\Current" name="codilar-testimonial-link">`	Adds a new `Link\Current` block named `codilar-testimonial-link`. This block type renders a link and, when the visitor is already on that page, shows it as plain highlighted text instead.

7-10	`                <arguments>`
	`                    <argument name="label" xsi:type="string" translate="true">Testimonials</argument>`
	`                    <argument name="path" xsi:type="string">testimonial</argument>`
	`                </arguments>`
		Arguments: `label` "Testimonials" (`translate="true"` runs it through the translation system) and `path` `testimonial`, which becomes the URL `/testimonial`.

11-14	`            </block>`
	`        </referenceBlock>`
	`    </body>`
	`</page>`
		Close the block, reference, body and page.

view/frontend/layout/testimonial_index_index.xml

> Layout for the testimonials page itself. The file name is the handle `{routeId}_{controller}_{action}`, so it only applies to `/testimonial`.

Lines	Code	Explanation

1-4	`<?xml version="1.0"?>`
	`<page xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"`
	`      layout="1column"`
	
	`xsi:noNamespaceSchemaLocation="urn:magento:framework:View/Layout/etc/page_configuration.xsd">`
		XML declaration and `<page>` root. `layout="1column"` uses the full-width page layout with no sidebars.

5-6	`    <body>` `        <referenceContainer name="content">`	Targets Magento's main `content` container (the central area of the page).
7	`            <container name="codilar.testimonial.page" htmlTag="div" htmlClass="testimonial-page">`	Adds a container that outputs a `<div class="testimonial-page">`. All the module's CSS is scoped under this class so it cannot affect other pages.
8	`                <!-- Not cacheable: the form refills typed values after errors and new entries must show at once. -->`	Comment explaining the next choice.

9-12	`                <block class="Codilar\Testimonial\Block\Form"`
	
	`name="codilar.testimonial.form"`
	
	`template="Codilar_Testimonial::form.phtml"`
	`                        cacheable="false"/>`
		The form block: class `Block\Form`, template `Codilar_Testimonial::form.phtml` (which means `view/frontend/templates/form.phtml` in this module). `cacheable="false"` excludes the **whole page** from Full Page Cache. That is needed because the page contains per-visitor data (refilled form values) and because new testimonials must appear immediately. The trade-off is that the page is rendered by PHP on every visit.

13-16	`                <block class="Codilar\Testimonial\Block\TestimonialList"`
	
	`name="codilar.testimonial.list"`
	
	`template="Codilar_Testimonial::list.phtml"`
	
	`after="codilar.testimonial.form"/>`
		The list block: class `Block\TestimonialList`, template `list.phtml`. `after="codilar.testimonial.form"` places it after the form.

17-20	`            </container>`
	`        </referenceContainer>`
	`    </body>`
	`</page>`
		Close the container, reference, body and page.

view/frontend/templates/form.phtml

> HTML for the "Write a Testimonial" form. `$block` is the `Block\Form` instance and `$escaper` is Magento's output escaper. Every dynamic value is escaped to prevent XSS.

Lines	Code	Explanation

1-7	`<?php`
	`/**`
	` * "Write a Testimonial" form.`
	` *`
	` * @var \Codilar\Testimonial\Block\Form $block`
	` * @var \Magento\Framework\Escaper $escaper`
	` */`
		Docblock with `@var` hints so IDEs know the types of `$block` and `$escaper`.

8-11	`$selectedRating = (int) $block->getFieldValue('rating');`
	`$honeypot = $block->getHoneypotFieldName();`
	`?>`
		Reads the previously selected rating (as an integer, 0 if none) and the honeypot field name into variables. Closes PHP.

12-13	`<div class="testimonial-form-wrapper">` `    <h2 class="testimonial-section-title"><?= $escaper->escapeHtml(__('Write a Testimonial')) ?></h2>`	Wrapper `div` and the section heading. `__()` translates and `escapeHtml()` escapes for HTML text.

14-18	`    <form class="form testimonial-form"`
	`          action="<?= $escaper->escapeUrl($block->getFormAction()) ?>"`
	`          method="post"`
	`          id="testimonial-form"`
	`          data-mage-init='{"validation":{}}'>`
		The `<form>`: posts to the URL from `getFormAction()` (escaped with `escapeUrl`). `data-mage-init='{"validation":{}}'` attaches Magento's jQuery validation widget, which checks the `data-validate` rules in the browser before the form is submitted.

19	`        <?= $block->getBlockHtml('formkey') ?>`	Outputs the hidden `<input name="form_key">` that the Save controller checks for CSRF protection.
20	`        <fieldset class="fieldset">`	Opens the fieldset that groups the fields.

21-29	`            <div class="field name required">`
	`                <label class="label" for="testimonial-name"><span><?= $escaper->escapeHtml(__('Name')) ?></span></label>`
	`                <div class="control">`
	`                    <input type="text" name="name" id="testimonial-name" class="input-text"`
	`                           maxlength="255" autocomplete="name"`
	`                           value="<?= $escaper->escapeHtmlAttr($block->getFieldValue('name')) ?>"`
	`                           data-validate="{required:true}"/>`
	`                </div>`
	`            </div>`
		Name: CSS class `required` adds the asterisk. `maxlength="255"` matches the database column, `autocomplete="name"` lets the browser autofill, the value is refilled through `getFieldValue('name')` (escaped for an attribute), and `{required:true}` is the client-side rule.

30-37	`<div class="field city">`
	`<label class="label" for="testimonial-city"><span><?= $escaper->escapeHtml(__('City')) ?></span></label>`
	`<div class="control">`
	`<input type="text" name="city" id="testimonial-city" class="input-text" maxlength="100" autocomplete="address-level2" value="<?= $escaper->escapeHtmlAttr($block->getFieldValue('city')) ?>"/>`
	`</div>`
	`</div>`
		City: optional. `maxlength="100"`, autocomplete hint `address-level2` (city), refilled after errors.

38-39	`<fieldset class="field rating required">` `<legend class="label"><span><?= $escaper->escapeHtml(__('Rating')) ?></span></legend>`	**Rating** uses a `<fieldset>` with a `<legend>`, the accessible way to label a group of radio buttons.

40-42	`<div class="control">`
	`<div class="testimonial-star-input">`
	`<?php for ($star = 5; $star >= 1; $star--): ?>`
		Star container. The loop counts **down** from 5 to 1. CSS then reverses the visual order with `row-reverse`, so the stars appear 1 to 5. Outputting them backwards lets the CSS sibling selector `~` highlight a star and every star to its left.

43-46	`<input type="radio" name="rating" value="<?= (int) $star ?>" id="testimonial-rating-<?= (int) $star ?>" <?= $selectedRating === $star ? 'checked' : '' ?> <?= $star === 1 ? 'data-validate="{\'validate-one-required-by-name\':true}"' : '' ?>/>`	One radio per star, with value and id from the loop and cast to int. It is pre-checked if it matches the refilled rating. Only the 1-star radio has the rule `validate-one-required-by-name`, which means "at least one radio named `rating` must be checked".

47-51	`<label for="testimonial-rating-<?= (int) $star ?>" title="<?= $escaper->escapeHtmlAttr(__('%1 out of 5 stars', $star)) ?>">`
	`<span class="sr-only"><?= $escaper->escapeHtml(__('%1 out of 5 stars', $star)) ?></span>`
	`</label>`
	`<?php endfor; ?>`
		The star label. CSS draws the star symbol. The `title` tooltip and the `sr-only` text ("3 out of 5 stars") are for mouse users and screen readers. End of loop.

52-54	`</div>`
	`</div>`
	`</fieldset>`
		Close the star container, control and rating fieldset.

55-64	`<div class="field message required">`
	`<label class="label" for="testimonial-message"><span><?= $escaper->escapeHtml(__('Message')) ?></span></label>`
	`<div class="control">`
	`<textarea name="message" id="testimonial-message" class="input-text" rows="5" maxlength="1000" data-validate="{required:true, minlength:20, maxlength:1000}" ><?= $escaper->escapeHtml($block->getFieldValue('message')) ?></textarea>`
	`<div class="note"><?= $escaper->escapeHtml(__('Between 20 and 1000 characters.')) ?></div>`
	`</div>`
	`</div>`
		Message: a `textarea` with `maxlength="1000"` and client-side rules matching the server (required, 20 to 1000 characters). The content between the tags is the refilled value. A `note` below explains the length rule.

65-72	`<div class="testimonial-hp" aria-hidden="true">`
	`<label for="testimonial-<?= $escaper->escapeHtmlAttr($honeypot) ?>">`
	`<?= $escaper->escapeHtml(__('Leave this field empty')) ?>`
	`</label>`
	`<input type="text" name="<?= $escaper->escapeHtmlAttr($honeypot) ?>"`
	`id="testimonial-<?= $escaper->escapeHtmlAttr($honeypot) ?>"`
	`value="" tabindex="-1" autocomplete="off"/>`
	`</div>`
		Honeypot: a text field named `website` moved off-screen by CSS. `aria-hidden` hides it from screen readers, `tabindex="-1"` keeps it out of keyboard focus and `autocomplete="off"` stops browsers filling it. Real visitors leave it empty.

73	`</fieldset>`	Closes the fieldset.

74-82	`<div class="actions-toolbar">`
	`<div class="primary">`
	`<button type="submit" class="action submit primary">`
	`<span><?= $escaper->escapeHtml(__('Submit Testimonial')) ?></span>`
	`</button>`
	`</div>`
	`</div>`
	`</form>`
	`</div>`
		Submit button in Magento's standard `actions-toolbar` markup so it picks up theme button styles. Close the form and wrapper.

view/frontend/templates/list.phtml

> HTML for the list of testimonial cards plus the pager.

Lines	Code	Explanation

1-8	`<?php`
	`/**`
	` * Testimonials list, newest first.`
	` *`
	` * @var \Codilar\Testimonial\Block\TestimonialList $block`
	` * @var \Magento\Framework\Escaper $escaper`
	` */`
		Docblock with type hints for `$block` (`TestimonialList`) and `$escaper`.

9-10	`$testimonials = $block->getTestimonials();` `?>`	Gets the collection (sorted, and limited to the current page by the pager). Closes PHP.
11-12	`<div class="testimonial-list-wrapper">` `    <h2 class="testimonial-section-title"><?= $escaper->escapeHtml(__('What Our Customers Say')) ?></h2>`	Wrapper and "What Our Customers Say" heading.

13-16	`    <?php if (!$testimonials->getSize()): ?>`
	`        <div class="message info empty">`
	`            <div><?= $escaper->escapeHtml(__('No testimonials yet. Be the first to share your experience!')) ?></div>`
	`        </div>`
		`getSize()` runs a `COUNT(*)` on the whole table. If it is 0, a standard Magento info message is shown instead of an empty list.

17-18	`    <?php else: ?>` `        <ol class="testimonial-list">`	Otherwise opens an ordered list (`<ol>`), since the entries are in date order.

19-21	`            <?php /** @var \Codilar\Testimonial\Model\Testimonial $testimonial */ ?>`
	`            <?php foreach ($testimonials as $testimonial): ?>`
	`                <?php $rating = (int) $testimonial->getData('rating'); ?>`
		Type hint for `$testimonial`, then the `foreach`, which is where the collection actually runs its `SELECT ... LIMIT 10`. Reads the rating as an integer.

22	`                <li class="testimonial-card">`	Each testimonial is a card (`<li>`).

23-29	`                    <div class="testimonial-stars"`
	`                         role="img"`
	`                         aria-label="<?= $escaper->escapeHtmlAttr(__('Rated %1 out of 5', $rating)) ?>">`
	`                        <?php for ($star = 1; $star <= 5; $star++): ?>`
	`                            <span class="star<?= $star <= $rating ? ' filled' : '' ?>" aria-hidden="true">★</span>`
	`                        <?php endfor; ?>`
	`                    </div>`
		Star display. The `div` has `role="img"` and an `aria-label` such as "Rated 4 out of 5", so screen readers announce one sentence. It outputs 5 star characters (`★` = ★); stars up to the rating get the class `filled` (orange), the rest stay grey. Each star is `aria-hidden`.

30-32	`                    <blockquote class="testimonial-message">` `                        <?= nl2br($escaper->escapeHtml($testimonial->getData('message'))) ?>` `                    </blockquote>`	The message in a `<blockquote>`. It is escaped **first**, then `nl2br()` turns line breaks into ` `. That order matters: reversing it would either escape the ` ` tags or allow injected HTML.
33-34	`                    <div class="testimonial-meta">` `                        <strong class="testimonial-name"><?= $escaper->escapeHtml($testimonial->getData('name')) ?></strong>`	Meta row: the customer name in bold.

35-37	`<?php if ($testimonial->getData('city')): ?>`
	`<span class="testimonial-city"><?= $escaper->escapeHtml($testimonial->getData('city')) ?></span>`
	`<?php endif; ?>`
		City, shown only when present (it can be NULL). CSS adds a "·" separator before it.

38-41	`<time class="testimonial-date"`
	`datetime="<?= $escaper->escapeHtmlAttr($testimonial->getData('created_at')) ?>">`
	`<?= $escaper->escapeHtml($block->getFormattedDate($testimonial)) ?>`
	`</time>`
		`<time>` with a machine-readable `datetime` attribute (raw DB value) and the human-readable localized date as its text.

42-45	`</div>`
	`</li>`
	`<?php endforeach; ?>`
	`</ol>`
		Close the meta row, card, loop and list.

46-48	`<?php if ($pagerHtml = $block->getPagerHtml()): ?>`
	`<div class="testimonial-toolbar toolbar bottom"><?= /* @noEscape */ $pagerHtml ?></div>`
	`<?php endif; ?>`
		If the pager has output, it is placed in a toolbar. `/* @noEscape */` marks this as intentionally unescaped: it is HTML produced by a Magento block, not user input, and escaping it would show the raw tags.

49-50	`<?php endif; ?>` `</div>`	Close the if/else and the wrapper.

view/frontend/web/css/source/_module.less

> Styles for the page. Magento's LESS build automatically includes any `_module.less` from a module's `web/css/source` folder in the theme's `styles-m.css` / `styles-l.css`. Variables such as `@indent__m` come from the Magento UI library, so spacing and colors follow the active theme.

Lines	Code	Explanation

1-4	`//`
	`//  Testimonials page (/testimonial)`
	`//  _____...`
		Section header comment, in Magento's LESS comment style.

5-6	`@testimonial-star__color: #ff5501;` `@testimonial-star-empty__color: #c7c7c7;`	Two module variables: filled star color (orange `#ff5501`) and empty star color (light grey).
7-8	`& when (@media-common = true) {`	`& when (@media-common = true)` is Magento's guard for styles that apply at every screen size. It makes sure the block is compiled once into `styles-m.css` rather than repeated.
9	`    .testimonial-page {`	Everything is nested under `.testimonial-page` (the container from the layout XML), so no styles leak onto other pages.

10-13	`        .testimonial-section-title {`
	`            margin: 0 0 @indent__m;`
	`            font-weight: @font-weight__light;`
	`        }`
		Section titles: bottom margin and a light font weight.

14-20	`        .testimonial-form-wrapper {`
	`            margin-bottom: @indent__xl;`
	`            padding: @indent__m;`
	`            border: 1px solid @border-color__base;`
	`            background: @color-white-smoke;`
	`        }`
		Form box: spacing below, padding, a 1px border and a light grey background to separate it from the list.

21-28	`        .testimonial-form {`
	`            max-width: 640px;`
	
	`            .fieldset {`
	`                margin-bottom: @indent__base;`
	`            }`
		Form at most 640px wide for readable line lengths; spacing under the fieldset.

29-33	`            .field.rating {`
	`                border: 0;`
	`                padding: 0;`
	`            }`
	`        }`
		Removes the default border and padding of the rating `<fieldset>`.

34-39	`        // Radios are output 5 → 1 and shown reversed, so ~ selects the lower stars.`
	`        .testimonial-star-input {`
	`            display: inline-flex;`
	`            flex-direction: row-reverse;`
		Star input container. `inline-flex` with `row-reverse` visually flips the 5-to-1 HTML order into 1-to-5 (see the comment on line 35).

40-46	`            input {`
	`                position: absolute;`
	`                opacity: 0;`
	`                width: 1px;`
	`                height: 1px;`
	`            }`
		Hides the actual radio circles but keeps them in the page (not `display:none`) so they still take keyboard focus and work with validation.

47-57	`label {`
	`  cursor: pointer;`
	`  padding: 0 2px;`
	`  font-size: 2.8rem;`
	`  line-height: 1;`
	`  color: @testimonial-star-empty__color;`
	
	`  &:before {`
	`  content: '\2605';`
	`  }`
	`  }`
		Each label is a large clickable grey star. `content: '\2605'` draws ★ with CSS.

58-64	`input:checked ~ label,`
	`label:hover,`
	`label:hover ~ label {`
	`  color: @testimonial-star__color;`
	`  }`
		The star-rating trick: the checked star, the hovered star and every `label` after them in the HTML (which, because of the reversal, appear to their **left**) turn orange.

65-68	`input:focus-visible + label {`
	`  outline: 2px solid @focus__color;`
	`  }`
	`  }`
		Keyboard accessibility: when a radio has keyboard focus, its star gets an outline.

69-73	`.sr-only {`
	`  .lib-visually-hidden();`
	`  }`
		`.sr-only` uses the UI library mixin `.lib-visually-hidden()`: invisible on screen but read by screen readers.

74-81	`.testimonial-hp {`
	`  position: absolute;`
	`  left: -10000px;`
	`  width: 1px;`
	`  height: 1px;`
	`  overflow: hidden;`
	`  }`
		Honeypot hiding: positioned 10000px off-screen and clipped to 1px. Bots reading the HTML still see the field.

82-90	`.testimonial-list {`
	`  display: grid;`
	`  grid-template-columns: 1fr;`
	`  gap: @indent__base;`
	`  margin: 0 0 @indent__m;`
	`  padding: 0;`
	`  list-style: none;`
	`  }`
		The list is a CSS grid with one column on mobile, a gap between cards and no bullets or numbers.

91-101	`.testimonial-card {`
	`  display: flex;`
	`  flex-direction: column;`
	`  margin: 0;`
	`  padding: @indent__m;`
	`  border: 1px solid @border-color__base;`
	`  border-radius: 4px;`
	`  background: @color-white;`
	`  box-shadow: 0 1px 3px rgba(0, 0, 0, .06);`
	`  }`
		Card: vertical flex layout, padding, a border with rounded corners, white background and a subtle shadow.

102-115	`.testimonial-stars {`
	`  margin-bottom: @indent__s;`
	`  font-size: 1.8rem;`
	`  line-height: 1;`
	
	`  .star {`
	`  color: @testimonial-star-empty__color;`
	
	`  &.filled {`
	`  color: @testimonial-star__color;`
	`  }`
	`  }`
	`  }`
		Star row on cards: size and spacing. Stars are grey by default and orange with the `.filled` class.

116-124	`        .testimonial-message {`
	`            flex: 1;`
	`            margin: 0 0 @indent__base;`
	`            padding: 0;`
	`            border: 0;`
	`            font-style: normal;`
	`            overflow-wrap: anywhere;`
	`        }`
		Message: `flex: 1` lets it grow so the meta rows of cards side by side line up at the bottom. Resets the browser's default blockquote styles. `overflow-wrap: anywhere` breaks very long words or URLs so they cannot widen the card.

125-134	`        .testimonial-meta {`
	`            display: flex;`
	`            flex-wrap: wrap;`
	`            align-items: baseline;`
	`            gap: 4px @indent__s;`
	`            padding-top: @indent__s;`
	`            border-top: 1px solid @border-color__base;`
	`            font-size: @font-size__s;`
	`            color: @text__color__muted;`
		Meta row: wrapping flex row with a top border, smaller muted text and small gaps.

135-138	`            .testimonial-name {`
	`                color: @text__color;`
	`            }`
		The name uses the normal (darker) text color so it stands out from the muted meta text.

139-143	`            .testimonial-city:before {`
	`                content: '\00B7';`
	`                margin-right: @indent__s;`
	`            }`
		Adds a middle dot (`\00B7` = ·) before the city as a separator.

144-149	`            .testimonial-date {`
	`                margin-left: auto;`
	`            }`
	`        }`
	`    }`
	`}`
		`margin-left: auto` pushes the date to the right end of the row. Close all open blocks.

150		Blank line.

151-155	`.media-width(@extremum, @break) when (@extremum = 'min') and (@break = @screen__m) {`
	`    .testimonial-page .testimonial-list {`
	`        grid-template-columns: repeat(2, 1fr);`
	`    }`
	`}`
		Magento's responsive mixin. This block applies only at widths from `@screen__m` (768px by default) upward and is compiled into `styles-l.css`. On tablets and desktops the list becomes two columns.

i18n/en_US.csv

> Translation dictionary for the `en_US` locale. Each line is `"original text","translation"`. Here both sides are the same, so it acts as a template: copy it to `ar_SA.csv`, `fr_FR.csv` and so on and translate the second column. `%1` and `%2` are placeholders filled by the arguments to `__()`.

Lines	Code	Explanation
1	`"Testimonials","Testimonials"`	Page title and footer link label (Index controller, `default.xml`).

2-6	`"Write a Testimonial","Write a Testimonial"`
	`"Name","Name"`
	`"City","City"`
	`"Rating","Rating"`
	`"Message","Message"`
		Form heading and field labels (`form.phtml`).

7	`"%1 out of 5 stars","%1 out of 5 stars"`	Star tooltip / screen-reader text, e.g. "3 out of 5 stars" (`form.phtml`).

8-10	`"Between 20 and 1000 characters.","Between 20 and 1000 characters."`
	`"Leave this field empty","Leave this field empty"`
	`"Submit Testimonial","Submit Testimonial"`
		Message length hint, honeypot label and submit button text (`form.phtml`).

11-13	`"What Our Customers Say","What Our Customers Say"`
	`"No testimonials yet. Be the first to share your experience!","No testimonials yet. Be the first to share your experience!"`
	`"Rated %1 out of 5","Rated %1 out of 5"`
		List heading, empty-list message and star aria-label (`list.phtml`).

14-16	`"Thank you for your testimonial!","Thank you for your testimonial!"`
	`"Invalid form key. Please refresh the page and try again.","Invalid form key. Please refresh the page and try again."`
	`"We could not save your testimonial right now. Please try again later.","We could not save your testimonial right now. Please try again later."`
		Success, invalid form key and save-failure messages (`Save.php`).

17-22	`"Please enter your name.","Please enter your name."`
	`"Name must be %1 characters or fewer.","Name must be %1 characters or fewer."`
	`"City must be %1 characters or fewer.","City must be %1 characters or fewer."`
	`"Please choose a rating between 1 and 5 stars.","Please choose a rating between 1 and 5 stars."`
	`"Please enter your message.","Please enter your message."`
	`"Your message must be between %1 and %2 characters.","Your message must be between %1 and %2 characters."`
		Validation error messages (`Save::validate()`).